



JF Aesthetics platform rebuild

Base Laravel 8 Hospital Management System

Repositioned as JF Aesthetics, Karachi

Prepared 21 Aug 2026

Everything shipped in this engagement so far: turning a stock Laravel hospital template into JF Aesthetics' clinic system — fixing a recurring class of schema-mismatch bugs, completing three features that were built but never wired up, adding invoice generation from scratch, and redesigning the admin dashboard.

14

Schema / logic bugs
fixed

13

New migrations written

1

New module —
Invoicing

30+

Routes verified live

01 — IDENTITY

Rebrand to JF Aesthetics

Name, phone, and address now flow from one settings source instead of being scattered across hardcoded strings.

Site identity Updated

`APP_NAME` set to **JF Aesthetics** — this alone re-labels every page title, the admin header, the login screen, and the footer credit, since they all read from it already.

Settings key mismatch Bug

The public site's phone and email never actually reflected what the admin `Settings` panel saved — the template read `phone` / `email`, while the form actually wrote `business_phone` / `business_email`.

Every edit an admin made was silently discarded.

Realigned the keys in `layouts/app.blade.php`. Settings changes now take effect immediately. Values set: **03327841753**, Defence Phase 5, Near Medicare Clinic, Karachi.

Logo & icon Replaced

The old logo/icon files were static leftovers from the original theme — literally the words "Life Care" baked into a PNG, disconnected from any setting.

Generated a new mark with PHP GD (teal roundel + wordmark, "JF" monogram favicon) and replaced both the live `storage/app/public` files and the static fallback images, so no old branding remains anywhere in the theme.

Homepage copy Updated

Hero typewriter line rewritten from "Welcome to Life Care / We Care Your Health" to **"Welcome to JF Aesthetics / We Care Your Beauty."**

02 — ROOT CAUSE

Sitewide broken images

Every uploaded image on the site — logos, patient photos, employee photos — was 404ing. One missing command, found and fixed.

Storage symlink never created Root cause

`public/storage` is supposed to be a symlink into `storage/app/public`. It had never been created on this install, so every `../storage/...` image reference across the entire site resolved to nothing.

Ran `php artisan storage:link`. This alone fixed logos, patient/doctor photos, and any future admin file uploads.

Dead attribute crash on the homepage Bug

Four places in `index.blade.php` read `$app->icon_logo_path` — an attribute that doesn't exist on any model. It only "worked" before because `$app` is a Laravel-reserved variable name (the framework auto-shares the container as `$app` in every view) that the original code happened to shadow with a Settings row. Removing that shadow assignment made the container itself resolve, and the page hard-crashed.

Replaced all four occurrences with a clean local `$iconValue` pulled from the Settings model.

Booking form & doctor dropdown

The homepage's "Book Appointment" form is the client's most visible complaint — it silently failed on every submission.

Appointment request failed on every submit Bug

The form saved to a `doctor` property that didn't match the actual database column, `doctor_id` — which is required with no default. Every submission threw a SQL error instead of saving.

Renamed the property to `doctor_id` throughout `Appointmentform.php`. Verified directly: a request now inserts cleanly.

Doctor dropdown showed blank entries Bug

The option list read `$doctor->name` — an attribute that doesn't exist on the doctor model (only `employee_id` does). Every option rendered with no visible label.

Switched to `$doctor->employ->name` via the existing relation, and added a "Choose Doctor" placeholder. Confirmed real names now render, e.g. *Hortense Zulauf*.

The schema-mismatch sweep

A recurring bug class throughout this codebase: a Livewire component quietly writing to a property name that doesn't match its real database column. Once the pattern was recognized, a full audit surfaced it repeatedly.

Nurses Completed

A full CRUD screen already existed — form, validation, photo upload, table — but its database table and Eloquent model had simply never been created. Every visit would have fatally errored.

Created the `nurses` migration and model, restored the sidebar link that had been commented out.

Admin Appointments Bug

Trailing spaces in array keys (`'intime '`, `'outtime '`) silently dropped both times on every save.

`nurse_id` had no column at all. `description` is required with no default, but was never collected —

every save threw a DB error. The doctor dropdown had the same blank-label bug as the public form.

Fixed the typos, added the `nurse_id` column and relation, added a description field, fixed the dropdown, restored the sidebar link.

Operation Reports Bug

A copy-paste error wrote the doctor's ID into the `status` column instead of the actual status value.

One-line fix: `'status' => $this->doctor` → `$this->status`.

Requested Appointments (admin edit) Bug

Edit/update referenced columns that don't exist on this table at all (`patient_id`, `intime`, `outtime`) instead of the real ones. The edit view didn't exist, and there was no Edit button anywhere in the UI — the feature was entirely unreachable.

Rewrote edit/update against the real columns, built the missing view, wired up a working Edit button.

Medicine Store (inventory) Gap

Tracked price, quantity, and an opaque code — but no name. There was no way to tell what any row actually was.

Added a `name` column end-to-end (migration, model, form, table), and cleaned up malformed leftover markup on the input fields.

Silent foreign-key typos Bug

`departments.hod_id`, `departments.block_id`, and `hods.doctor_id` all used a typo'd Blueprint method (`constrand()` / `constraned()`) — Laravel accepted it as a harmless no-op instead of erroring, so no foreign key constraint was ever actually created.

Verified no orphaned rows existed, then added the real constraints via a new migration.

User role column Bug

The User model's fillable list included `role`; the actual column is `role_id`.

Corrected the fillable list.

Invoice generation

Built from nothing, against a reference invoice the client supplied: line items with per-item discount (flat or percentage), a running payment history, and a printable A4 page.

Data model

Three new tables — `invoices`, `invoice_items`, `invoice_payments` — support a patient with multiple service line items and multiple payments recorded over time, rather than the single flat amount the old `bills` table allowed.

Admin flow

Pick a patient, add line items (service, quantity, service charge, discount), add payment entries (date, amount, mode) — Grand Total, Paid, and Unpaid recompute live as you type.

Percentage discount

Each line item's discount can be entered as a flat `pkṛ` amount or a `%` — the percentage is computed against that line's subtotal automatically and shown on the invoice, e.g. *pkṛ 4,000 (10%)*.

A4 printable invoice

A dedicated print page matches the client's reference layout: clinic branding, patient name and MR#, invoice ID / date / printed-by, the item table with discount, the payment history table, and a totals summary — opens in a new tab with a print / save-as-PDF control.

Checked against the reference sample: computed Grand Total / Paid / Unpaid matched exactly — **40,000 / 20,000 / 20,000**.

06 — ADMIN DASHBOARD

Dashboard redesign

The client shared a dashboard from an unrelated retail/inventory system as a style reference. Rather than bolting on Product/Purchase/Sale modules that don't belong in a clinic system, the brief was clarified to: match the look, keep the real data.

What it shows now

Period-filterable summary cards (Today / Last 7 Days / This Month / This Year) for Revenue, Outstanding, New Patients, and Appointment Requests; a 7-month Cash Flow chart; a current-month Collected-vs-Outstanding donut; a 12-month Yearly Report; tabbed Recent Transactions with status badges; and Top Services tables — the clinic-relevant read of "best sellers."

Chart.js load-order bug Bug

The theme's bundled Chart.js (v2.9.4) was included at the very bottom of the page — after the dashboard's own chart-drawing scripts had already tried and failed to run, leaving every chart blank with no visible error.

Moved the include into `<head>` so the library is ready before any chart initializes.

07 — VERIFICATION

How it was actually tested

Every fix in this report was checked against a running instance of the app, not just read for correctness.

METHOD	WHAT IT COVERED
Authenticated route sweep	Logged in via a real session cookie and hit every admin sidebar route plus every public route — confirmed 200s across the board, none of the earlier 500s.
Tinker execution	Directly exercised the fixed create/update logic — booking form, admin appointments, nurses, medicine store, invoices with % discounts — confirming correct writes and computed totals.
Numeric cross-check	Invoice totals computed by the new module matched the client's reference sample invoice exactly.
Cleanup	All test and seed records created during verification were deleted afterward — nothing fake left in the database.

08 — FOR AWARENESS

Open items & environment

Disk space — outside the app, worth acting on

The machine's C: drive is completely full (120 GB / 120 GB used). Unrelated to this project, but it caused several tool and install failures mid-session and will cause worse problems left alone.

Left as-is, not treated as a flaw

Doctors admin page — the standalone `admin_doctors` route stays disabled by design; doctor records are created through Employees (position = doctor), which already works.

Doubled route paths — `hods` and `requestedappointments` resolve to `/admin/admin/...` due to an extra prefix in their route definitions. Cosmetic only — the app always calls them by route name, never the raw URL.